Appendix. Building Your Own Kubernetes Cluster

While Kubernetes is often experienced through the virtual world of public cloud computing, where the closest you get to your cluster is a web browser or a terminal, it can be a very rewarding experience to physically build a Kubernetes cluster on bare metal. Likewise, nothing compares to physically pulling the power or network on a node and watching how Kubernetes reacts to heal your application to convince you of its utility.

Building your own cluster might seem like both a challenging and an expensive effort, but fortunately it is neither. The ability to purchase low-cost, system-on-chip computer boards, as well as a great deal of work by the community to make Kubernetes easier to install, means that it is possible to build a small Kubernetes cluster in a few hours.

In the following instructions, we focus on building a cluster of Raspberry Pi machines, but with slight adaptations, the same instructions could be made to work with a variety of different single-board machines or any other computers you may have around.

Parts List

The first thing you need to do is assemble the pieces for your cluster. In all the examples here, we assume a four-node cluster. You could build a cluster of three nodes, or even a cluster of a hundred nodes if you wanted to, but four is a pretty good number. To start, you'll need to purchase (or scrounge) the various pieces needed to build the cluster.

Here is the shopping list, with some approximate prices as of the time of writing:

1. Four Raspberry Pi 4 machines with at least 2 GB of memory—$180
2. Four SDHC memory cards, at least 8 GB (buy high-quality ones!)—$30-50
3. Four 12-inch Cat. 6 Ethernet cables—$10
4. Four 12-inch USB-A to USB-C cables—$10
5. One 5-port 10/100 fast Ethernet switch—$10
6. One 5-port USB charger—$25

7. One Raspberry Pi stackable case capable of holding four Pis—$40 (or build your own)
8. One USB-to-barrel plug for powering the Ethernet switch (optional)—$5

The total for the cluster comes to about $300, which you can drop down to $200 by building a three-node cluster and skipping the case and the USB power cable for the switch (though the case and the cable really clean up the whole cluster).

One other note on memory cards: do not scrimp here. Low-end memory cards behave unpredictably and make your cluster really unstable. If you want to save some money, buy a smaller, high-quality card. High-quality 8 GB cards can be had for around $7 each online.

Once you have your parts, you're ready to move on to building the cluster.

NOTE

These instructions also assume that you have a device capable of flashing an SDHC card. If you do not, you will need to purchase a USB memory card reader/writer.

Flashing Images

The default Ubuntu 20.04 image supports Raspberry Pi 4 and also is a common operating system used by many Kubernetes clusters. The easiest way to install that is using the Raspberry Pi Imager provided by the Raspberry Pi project:

- macOS
- Windows
- Linux

Use the imager to write the Ubuntu 20.04 image onto each of your memory cards. Ubuntu may not be the default image choice in the imager, but you can select it as an option.

First Boot

The first thing to do is to boot just your API server node. Assemble your cluster, and decide which is going to be the API server node. Insert the memory card, plug the board

into an HDMI output, and plug a keyboard into the USB port.

Next, attach the power to boot the board.

Log in at the prompt using the username **ubuntu** and the password **ubuntu**.

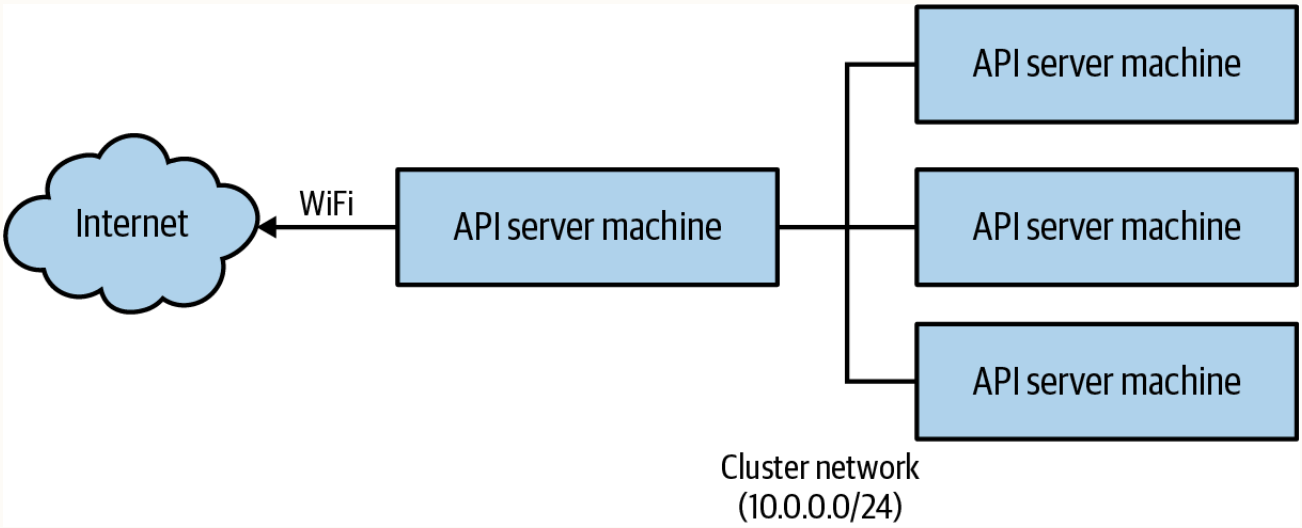
WARNING

The very first thing you should do with your Raspberry Pi (or any new device) is to change the default password. The default password for every type of install everywhere is well known by people who will misbehave given a default login to a system. This makes the internet less safe for everyone. Please change your default passwords!

Repeat these steps for each of the nodes in your cluster.

Setting Up Networking

The next step is to set up networking on the API server. Setting up networking for a Kubernetes cluster can be complicated. In the following example, we are setting up a network where a single machine is attached to the internet using wireless networking; this machine is also connected to a cluster network over wired Ethernet and provides a DHCP (Dynamic Host Configuration Protocol) server to provide a network address to the remaining nodes in the cluster. An illustration of this network is shown here:



Decide which of your boards will host the API server and `etcd`. It's often easiest to remember which one this is by making it the top or bottom node in your stack, but some sort of label also works.

To do this, edit the file */etc/netplan/50-cloud-init.yaml*. If this file doesn't exist, you can create it. The contents of the file should look like:

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: false
      dhcp6: false
      addresses:
      - '10.0.0.1/24'
      optional: true
  wifis:
    wlan0:
      access-points:
        <your-ssid-here>:
          password: '<your-password-here>'
      dhcp4: true
      optional: true
```

This sets the main Ethernet interface to have the statically allocated address 10.0.0.1 and sets up the WiFi interface to connect to your local WiFi. You should then run `sudo netplan apply` to pick up these new changes.

Reboot the machine to claim the 10.0.0.1 address. You can validate that this is set correctly by running `ip addr` and looking at the address for the `eth0` interface. Also validate that the connection to the internet works correctly.

Next, we're going to install DHCP on this API server so it will allocate addresses to the worker nodes. Run:

```
$ apt-get install isc-dhcp-server
```

Then configure the DHCP server as follows (*/etc/dhcp/dhcpd.conf*):

```
# Set a domain name, can basically be anything
option domain-name "cluster.home";

# Use Google DNS by default, you can substitute ISP-supplied values here
option domain-name-servers 8.8.8.8, 8.8.4.4;
```

```
# We'll use 10.0.0.X for our subnet
subnet 10.0.0.0 netmask 255.255.255.0 {
   range 10.0.0.1 10.0.0.10;
   option subnet-mask 255.255.255.0;
   option broadcast-address 10.0.0.255;
   option routers 10.0.0.1;
}
default-lease-time 600;
max-lease-time 7200;
authoritative;
```

You may also need to edit */etc/default/isc-dhcp-server* to set the `INTERFACES` environment variable to `eth0`. Restart the DHCP server with `sudo systemctl restart isc-dhcp-server`. Now your machine should be handing out IP addresses. You can test this by hooking up a second machine to the switch via Ethernet. This second machine should get the address 10.0.0.2 from the DHCP server.

Remember to edit the */etc/hostname* file to rename this machine to `node-1`. To help Kubernetes do its networking, you also need to set up `iptables` so that it can see bridged network traffic. Create a file at */etc/modules-load.d/k8s.conf* that just contains `br_netfilter`. This will load the `br_netfilter` module into your kernel.

Next you need to enable some `systemctl` settings for network bridging and address translation (NAT) so that Kubernetes networking will work, and your nodes can reach the public internet. Create a file named */etc/sysctl.d/k8s.conf* and add:

```
net.ipv4.ip_forward=1
net.bridge.bridge-nf-call-ip6tables=1
net.bridge.bridge-nf-call-iptables=1
```

Then edit */etc/rc.local* (or the equivalent) and add `iptables` rules for forwarding from `eth0` to `wlan0` (and back):

```
iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE
iptables -A FORWARD -i wlan0 -o eth0 -m state \
  --state RELATED,ESTABLISHED -j ACCEPT
iptables -A FORWARD -i eth0 -o wlan0 -j ACCEPT
```

At this point, the basic networking setup should be complete. Plug in and power up the remaining two boards (you should see them assigned the addresses 10.0.0.3 and

10.0.0.4). Edit the */etc/hostname* file on each machine to name them `node-2` and `node-3`, respectively.

Validate this by first looking at */var/lib/dhcp/dhcpd.leases*, and then SSH to the nodes (remember again to change the default password first thing). Validate that the nodes can connect to the external internet.

EXTRA CREDIT

There are a couple of extra steps you can take that will make it easier to manage your cluster. The first is to edit */etc/hosts* on each machine to map the names to the right addresses. On each machine, add:

```
...
10.0.0.1 kubernetes
10.0.0.2 node-1
10.0.0.3 node-2
10.0.0.4 node-3
...
```

Now you can use those names when connecting to those machines.

The second is to set up passwordless SSH access. To do this, run `ssh-keygen` and then copy the *$HOME/.ssh/id_rsa.pub* file into */home/ubuntu/.ssh/authorized_keys* on `node-1`, `node-2`, and `node-3`.

Installing a Container Runtime

Before you can install Kubernetes, you need to install a container runtime. There are several possible runtimes you can use, but the most broadly adopted is `containerd` from Docker. `containerd` is provided by the standard Ubuntu package manager, but its version tends to lag a little bit. It's a little more work, but we recommend installing it from the Docker project itself.

The first step is to set up Docker as a repository for installing packages on your system:

```
# Add some prerequisites
sudo apt-get install ca-certificates curl gnupg lsb-release
```

```
# Install Docker's signing key
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor \
-o /usr/share/keyrings/docker-archive-keyring.gpg
```

As a final step, create the file */etc/apt/sources.list.d/docker.list* with the following contents:

```
deb [arch=arm64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
https://download.docker.com/linux/ubuntu   focal stable
```

Now that you have installed the Docker package repository, you can install `containerd.io` by running the following command. It is important to install `containerd.io`, not `containerd`, to get the Docker package instead of the default Ubuntu package:

```
sudo apt-get update; sudo apt-get install containerd.io
```

At this point, `containerd` is installed, but you need to configure it since the configuration supplied by the package won't work with Kubernetes:

```
containerd config default > config.toml
sudo mv config.toml /etc/containerd/config.toml

# Restart to pick up the config
sudo systemctl restart containerd
```

Now that you have a container runtime installed, you can move on to installing Kubernetes itself.

Installing Kubernetes

At this point you should have all nodes up with IP addresses and capable of accessing the internet. Now it's time to install Kubernetes on all of the nodes. Using SSH, run the following commands on all nodes to install the `kubelet` and `kubeadm` tools.

First, add the encryption key for the packages:

```
# curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg \
| sudo apt-key add -
```

Then add the repository to your list of repositories:

```
# echo "deb http://apt.kubernetes.io/ kubernetes-xenial main" \
  | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

Finally, update and install the Kubernetes tools. This will also update all packages on your system for good measure:

```
# sudo apt-get update
$ sudo apt-get upgrade
$ sudo apt-get install -y kubelet kubeadm kubectl kubernetes-cni
```

Setting Up the Cluster

On the API server node (the one running DHCP and connected to the internet), run:

```
$ sudo kubeadm init --pod-network-cidr 10.244.0.0/16 \
      --apiserver-advertise-address 10.0.0.1 \
      --apiserver-cert-extra-sans kubernetes.cluster.home
```

Note that you are advertising your internal-facing IP address, not your external address.

Eventually, this will print out a command for joining nodes to your cluster. It will look something like:

```
$ kubeadm join --token=<token> 10.0.0.1
```

SSH onto each of the worker nodes in your cluster and run that command.

When all of that is done, you should be able to run this command and see your working cluster:

```
$ kubectl get nodes
```

Setting Up Cluster Networking

You have your node-level networking set up, but you still need to set up the Pod-to-Pod networking. Since all of the nodes in your cluster are running on the same physical Ethernet network, you can simply set up the correct routing rules in the host kernels.

The easiest way to manage this is to use the Flannel tool created by CoreOS and now supported by the Flannel project. Flannel supports a number of different routing modes; we will use the `host-gw` mode. You can download an example configuration from the Flannel project page:

```
$ curl https://oreil.ly/kube-flannelyml \
  > kube-flannel.yaml
```

The default configuration that Flannel supplies uses `vxlan` mode instead. To fix this, open up that configuration file in your favorite editor; replace `vxlan` with `host-gw`.

You can also do this with the `sed` tool in place:

```
$ curl https://oreil.ly/kube-flannelyml \
  | sed "s/vxlan/host-gw/g" \
  > kube-flannel.yaml
```

Once you have your updated *kube-flannel.yaml* file, you can create the Flannel networking setup with:

```
$ kubectl apply -f kube-flannel.yaml
```

This will create two objects, a ConfigMap used to configure Flannel and a DaemonSet that runs the actual Flannel daemon. You can inspect these with:

```
$ kubectl describe --namespace=kube-system configmaps/kube-flannel-cfg
$ kubectl describe --namespace=kube-system daemonsets/kube-flannel-ds
```

Summary

At this point, you should have a working Kubernetes cluster operating on your Raspberry Pis. This can be great for exploring Kubernetes. Schedule some jobs, open up the UI, and try breaking your cluster by rebooting machines or disconnecting the network.

Previous chapter
< 22. Organizing Your Application

Next chapter
Index >